

Blockchain Overview

This note compiles lecture-scale PDFs from the IIT Kharagpur NPTEL series, offering a concise roadmap of blockchain fundamentals—from decentralization and cryptographic building blocks to mining, consensus mechanisms, and emerging alternatives to proof-of-work.

Content Sources

- Section 1: Summarizes **Beyond PoW**, **Consensus**, **Scaling** from a 96-page PDF presentation. ([source](#))
- Section 2: Details **P2P network**, **Transaction validation**, **Forking** from a 117-page PDF presentation. ([source](#))
- Section 3: Explores **Bitcoin mining**, **Economic model**, **Cryptocurrency** from a 105-page PDF presentation. ([source](#))
- Section 4: Covers **Cryptographic primitives**, **Distributed consensus**, **Cryptocurrency design** from a 112-page PDF presentation. ([source](#))
- Section 5: Introduces **Decentralization**, **Blockchain model**, **Foundations** from a 106-page PDF presentation. ([source](#))

Section 1: Beyond Proof-of-Work – Alternative Consensus Mechanisms

This section covers open consensus mechanisms beyond Proof-of-Work, including PoS, PoB, and PoET, from a 96-page lecture presentation ([source](#))

Opening Context

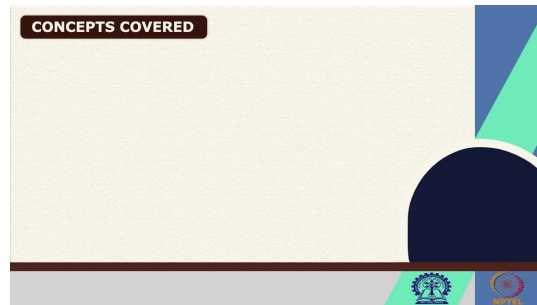
The lecture begins with promotional material for NPTEL online certification courses from IIT Kharagpur.



The image shows the IIT Kharagpur building and the "NPTEL ONLINE CERTIFICATION COURSES" banner, reminding viewers of the source of the material.

Open Consensus Mechanisms (Beyond PoW)

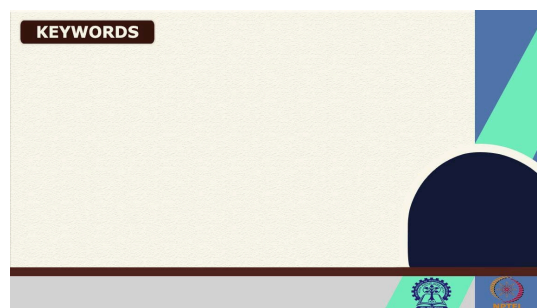
Consensus Variant	Core Idea
Proof-of-Stake (PoS)	Mining power proportional to the amount of cryptocurrency held
Proof-of-Burn (PoB)	Miners destroy (send to an un-spendable address) a portion of their coins to gain the right to mine
Proof-of-Elapsed-Time (PoET)	Participants wait a randomly-chosen time; the first to finish creates the next block



The Limits of Proof-of-Work

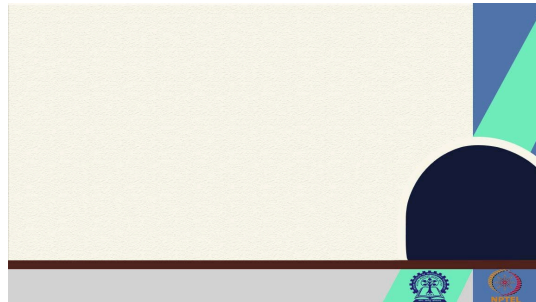
The Good

- Fully **decentralized consensus** for permissionless networks
- Works well for cryptocurrencies (e.g., Bitcoin, Ethereum)



The Bad

- Requires a **huge network** to mitigate 51% attacks → **impractical for enterprise use**



The Ugly

- **Very low transaction throughput**
- **Massive energy consumption**

Throughput examples:

Blockchain	TPS
Bitcoin	3.3 – 7 TPS
Ethereum (pre-2.0)	≈ 15 TPS

Energy figures (Digiconomist):

Metric	Value	Equivalent
Carbon footprint	825.47 kg CO ₂ per transaction	137,578 hours of YouTube streaming
Electrical energy	1,737.82 kWh per transaction	U.S. household for 59.56 days

🌐 Proof-of-Stake (PoS)

Historical note: Idea possibly first posted in [2011 on BitcoinTalk](#) (topic 27787)

Transition logic: Move from PoW to PoS once bitcoins are widely distributed

PoW vs PoS Comparison

Metric	PoW	PoS
Mining probability	Proportional to computational work performed	Proportional to stake (percentage of total coins held)

Benefits of PoS

- **Higher economic protection:** Attacking requires owning a large fraction of the total coin supply
- **Attack cost is borne by the attacker:** The attack reduces the value of the attacker's own stake

Stake Variants

Variant	Description	Examples
Randomized stake	Combines stake amount with a random factor	Nxt, BlackCoin
Coin-age	$\text{Stake} = \text{coins} \times \text{days held}$	Peercoin

🔥 Proof-of-Burn (PoB)

- Miners **burn** coins by sending them to an **un-spendable address**, proving commitment

Cost comparison:

Mechanism	Resource Type
PoW	Real resource → electricity, hardware
PoB	Virtual/digital resource → permanently destroyed cryptocurrency

Bootstrap limitation: PoB depends on an existing PoW-minted coin; cannot start a brand-new blockchain without an existing supply

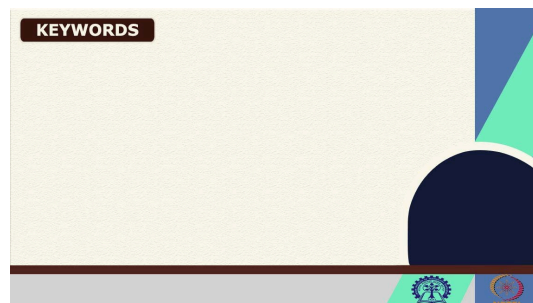
🕒 Proof-of-Elapsed-Time (PoET)

- Proposed by [Intel](#) as part of the [Hyperledger Sawtooth](#) platform

Basic idea: Each node waits a random time; the first node whose timer expires becomes the block proposer

Verification via Intel SGX

- Uses **Software Guard Extensions (SGX)** enclave to prove that the waiting period really elapsed
- SGX provides **hardware-based attestation** that the trusted code executed correctly



🎓 Conclusion (Lecture 21)

Mechanism	Key Characteristic
PoW	Extremely costly in energy and hardware
PoS	Reduces cost by leveraging stake
PoB	Reduces cost by burning coins
PoET	Low-cost consensus but requires specialized hardware (SGX)

Open question: Which of these mechanisms is most suitable for enterprise blockchains?

Section 2: Ethereum Foundations and Geth Client

This section covers Ethereum fundamentals, the Go Ethereum (Geth) client, account management, network connections, and JSON-RPC interactions from a 117-page lecture presentation ([source](#))

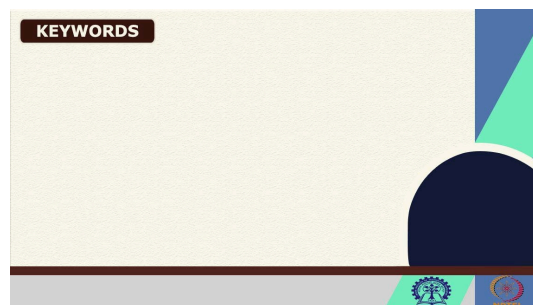
Introductory Overview

Ethereum = platform for sending cryptocurrency (Ether) **and** executing **smart contracts**

- **Permissionless** blockchain that can host **decentralized applications (DApps)**
- Official website: <https://ethereum.org/en/what-is-ethereum/>

Ethereum Network Basics

Property	Description
Nodes	Computers running an Ethereum client; each node can verify blocks & transactions
Topology	Peer-to-peer (P2P) system with no central server



Mainnet resources:

- <https://ethstats.net/>
- <https://www.ethernodes.org/countries>

Go Ethereum (Geth) – The Official Go Client

Feature	Description
Implementation	Go language implementation of the Ethereum protocol
Node types	• Full node (default) – stores full blockchain data • Archive node – retains every historical state • Light node – fetches data on-demand
APIs	Exposes JSON-RPC over HTTP , WebSocket , and IPC transports

Installing Geth on Ubuntu

```
sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install ethereum
```

Managing Ethereum Accounts (CLI)

```
geth account command [options] [args...]
```

Sub-command	Purpose
list	Show all existing accounts
new	Create a new account (generates a keystore file)
update	Change account password
import	Import a raw private key into a new account

Create an account: `geth account new` **List accounts:** `geth account list` (keystore files stored under `~/.ethereum/keystore/`)

Connecting to Networks

Configuration		Result
Default (no flags)		Connects to Ethereum mainnet
Testnets (selected via flags)		Various test environments
Flag	Testnet	Consensus
<code>--ropsten</code>	Ropsten (PoW)	https://ropsten.etherscan.io/
<code>--rinkeby</code>	Rinkeby (PoA)	https://rinkeby.etherscan.io/
<code>--goerli</code>	Goerli (PoA)	https://goerli.etherscan.io/

Sync Modes

```
geth --syncmode ""
```

Mode	Behaviour
full	Download every block & execute all transactions
fast	Download headers, verify them, then download the state snapshot
snap (default)	Faster variant of fast
light	Download only block headers; request needed data on-demand

Example: Connecting to Goerli (light)

```
geth --goerli --syncmode "light"
```

Copying an account to the testnet keystore:

```
cp ~/.ethereum/keystore/UTC--2021-10-26T20-23-14.304161080Z--8010319e8e3115ea522dc2f00ab9bc28a5abbf80 \
~/.ethereum/rinkeby/keystore/
```

Interacting with Geth

Access method	Advantages	Limitations
IPC (JavaScript console)	Full node functionality (account creation, signing, etc.)	Requires direct file-system access to the node
HTTP RPC	Remote access for applications (e.g., DApp front-ends)	Limited to safe RPC calls; may expose attack surface

Enable HTTP RPC:

```
geth --goerli --syncmode "light" --http
```

JSON-RPC Example – Query Balance

Request payload (eth_getBalance):

```
{
  "jsonrpc": "2.0",
  "method": "eth_getBalance",
  "params": ["0x9808f22453Ee87cc23eA76ca7Ed66a4F294d54D4", "latest"],
  "id": 1
}
```

cURL command:

```
curl -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
--data '{"jsonrpc":"2.0","method":"eth_getBalance","params":["0x5342722156fcd4b0e0b140c4fb9cd63dcea347f4", "latest"]}'
```

Conclusion (Lecture 22)

Aspect	Key Takeaway
Ethereum's core capability	Permissionless, smart-contract platform
Geth	Primary CLI client (full/archival/light node types)
Network distinction	Mainnet vs testnets (Ropsten, Rinkeby, Goerli)
Account management	Via CLI commands (new, list, import, etc.)
Remote interaction	JSON-RPC over HTTP for querying node data

Section 3: Ethereum Transactions, Gas, and Testnet Operations

This section covers Ethereum transaction fundamentals, the gas mechanism, testnet faucets, and practical transaction operations from a 105-page lecture presentation ([source](#))

Transaction Basics

Transaction: A cryptographically signed instruction that changes the Ethereum state

- The simplest transaction = transfer of Ether from one account to another

Requirements for executing a transaction:

Requirement	Purpose
-------------	---------

Private key of the sending account	To cryptographically sign the transaction
Gas	The fee paid for computational resources

Gas Concept

Term	Definition
Gas	Unit measuring the computational effort of a transaction
Gas price	Price per gas unit, measured in wei (smallest Ether unit)
Gas limit	Maximum gas units willing to be consumed

Total fee calculation: Gas price × Gas limit = total fee in Ether

Obtaining Test-net Ether (Faucets)

Testnet	Faucet URLs
Goerli	- Social faucet: https://faucet.goerli.mudit.blog/ - Simple faucet: https://goerli-faucet.slock.it/
Rinkeby	https://faucet.rinkeby.io/ (requires social media post)
Ropsten	https://faucet.ropsten.be/ (PoW testnet)

Section 4: Bitcoin Network Fundamentals and Mining Mechanics

This section examines Bitcoin P2P network operations, transaction lifecycle, mining processes, block propagation, and fork handling from a 112-page lecture presentation ([source](#))

Bitcoin P2P Network Fundamentals

Property	Description
Topology	Ad-hoc, random ; runs over TCP
Node Equality	All nodes are treated equally (no privileged servers)
Dynamic Membership	New nodes may join anytime; non-responsive nodes are pruned after 3 hours

How a New Node Joins

Step	Action
1	Obtain a seed-node address – the entry point to the network
2	Contact the seed node (often a hard-coded list of IPs)
3	Download the most recent blockchain (headers & blocks)
4	Start sending/receiving transactions (become a full participant)

The slide showing the steps is rendered on the *Joining a Bitcoin P2P Network* background image.

Transaction Lifecycle

Stage	Description
Creation	Example: Alice → Bob, A → B: BTC 10
Script Inclusion	Alice attaches the unlocking/locking scripts

Broadcast (Flooding)	Alice's client sends the transaction to all peers
----------------------	---

Validation & Relaying Rules

- **Validity** – Must be consistent with the current blockchain (no double-spend, correct signatures)
- **Script Whitelist** – Only scripts from a pre-approved list are accepted (prevents infinite loops)
- **Conflict-Free** – The transaction must not clash with other already-relayed transactions

Transaction pool illustration (textual)

```
A→B:BTC10
A→B:BTC10
...
C→D:BTC20
C→D:BTC20
...
```

Nodes accept the first copy they receive and discard duplicates.

Mining & Block Generation

Aspect	Description
Miner's Role	Gather all received (valid) transactions, assemble a candidate block, and start the Proof-of-Work (PoW) puzzle
Puzzle Solution	First miner to find a nonce that yields a hash below the difficulty target generates a new block
Block Flooding	The winning miner broadcasts the block to the network (gossip protocol)

Block Propagation & Forks

- **Temporary forks** – Simultaneous block discoveries create divergent chains
- **Longest-chain rule** – The branch with the most accumulated proof-of-work becomes the canonical chain

Longest-chain illustration (textual)

```
Block 1  Block 2  ...  Block 10
Block 6  Block 7  ...  Block 11  ← longest
```

- **Accidental forks** are resolved when one branch overtakes the other
- **Intentional forks** (hard/soft) produce new network versions (e.g., Bitcoin Cash, protocol upgrades)

Selecting a Block to Relay

A node relays a block **only if**:

1. The block's hash correctly links to the current chain
2. All included transactions are valid (scripts, signatures)
3. The block resides on the **current longest chain** (fork blocks are ignored)

Lecture 16 Recap

Topic	Key Points
Network joining	Mechanism for a new node to join the Bitcoin network
Transactions	Creation, validation, and network flooding
Mining	Process, block propagation, and handling of forks

Consensus	Longest-chain rule for resolving conflicts
-----------	--

Section 5: Cryptographic Foundations and Distributed Consensus

This section covers cryptographic hash functions, public-key cryptography, RSA algorithm, digital signatures, and distributed consensus theory from a 106-page lecture presentation ([source](#))

Foundations of Cryptography

Cryptographically Secure Hash Functions

Property	Explanation
Collision-free	Two different inputs should not produce the same hash
Information hiding	The hash reveals no useful information about the original data
Puzzle-friendly	Hard to invert; suitable for proof-of-work puzzles
Hash pointers	Store both data and its hash, enabling tamper-evident structures
Hashchain	Sequential chaining of hashes (e.g., in block headers)
Merkle (hash) tree	Hierarchical hash aggregation allowing efficient proof of inclusion

Image – NPTEL promotional graphic

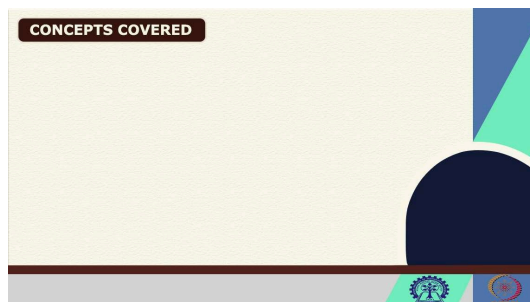


The slide header shows the IIT Kharagpur building and NPTEL branding, introducing the lecture series.

Symmetric vs. Asymmetric (Public-Key) Cryptography

Type	Characteristics	Blockchain Suitability
Symmetric key cryptography	Same secret key for encryption & decryption	Limited – secure key distribution is challenging; lacks public verifiability
Public-key (asymmetric) cryptography	Distinct <i>public</i> (encryption) and <i>private</i> (decryption) keys	Ideal – enables open verification, digital signatures, and decentralized trust

Image – "Concepts Covered" slide



Illustrates the agenda: basic concepts, public-key cryptography, RSA, and digital signatures.

🔑 Public-Key Cryptography in Detail

Core Definitions

Term	Definition
Key	Parameter that determines the output of a cryptographic algorithm
Encryption	$M' = E(M, k)$ – transforms plaintext M into ciphertext M' using key k
Decryption	$M = D(M', k)$ – restores the original message

Desired Key Properties

1. **True randomness** – Prevents guessing
2. **Sufficient length** – Larger key size → higher brute-force difficulty
3. **High entropy** – All bits equally random

🏰 RSA (Rivest-Shamir-Adleman) Algorithm

High-Level Description

- **Public key:** (e, n)
- **Private key:** (d, n)
- Anyone can **encrypt** with (e, n) ; only the holder of d can **decrypt**

RSA Algorithm Phases

Phase	Activity
1. Key Generation	Generate public and private key pair
2. Key Distribution	Share public key securely
3. Encryption	Transform plaintext to ciphertext
4. Decryption	Recover plaintext from ciphertext

Mathematical Foundation

Step	Computation
Choose primes	Select distinct large primes p and q
Compute modulus	$n = p \times q$
Compute Euler's totient	$\phi(n) = (p-1)(q-1)$
Select public exponent	Choose e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
Compute private exponent	$d \equiv e^{-1} \pmod{\phi(n)}$, i.e., $d \times e \equiv 1 \pmod{\phi(n)}$

Encryption: $c \equiv m^e \pmod{n}$

Decryption: $m \equiv c^d \pmod{n}$

Example (Small-Scale Illustration)

Step	Computation	Result
Choose primes	$p = 17, q = 11$	–
Modulus	$n = pq = 187$	–
Totient	$\phi(n) = (p-1)(q-1) = 160$	–
Public exponent	$e = 7$ (coprime with 160)	–

Private exponent	Find d such that $d \times 7 \equiv 1 \pmod{160} \rightarrow d = 23$	–
Encrypt M = 88	$c = 88^7 \pmod{187} = 11$	Ciphertext = 11
Decrypt C = 11	$m = 11^{23} \pmod{187} = 88$	Recovered plaintext = 88

Interactive RSA demo: <https://www.devglan.com/online-tools/rsa-encryption-decryption>

Digital Signatures

Goal	Description
Authentication	Verify that a document originates from the claimed sender
Integrity	Detect any alteration of the document
Non-repudiation	Sender cannot deny having signed the document
Uniqueness	Signature is bound to a specific document; cannot be transferred

Basic Process

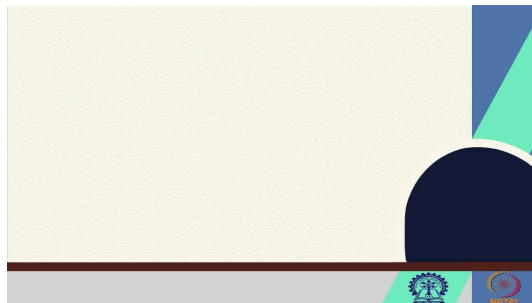
Step	Action	Formula
1. Signing	Sender encrypts message hash with private key	$S = E(H(M), K_{pri})$
2. Verification	Anyone decrypts S using public key , checks match	$H(M) \stackrel{?}{=} D(S, K_{pub})$

Optimization: To **reduce signature size**, the hash $H(M)$ (message digest) is signed instead of the full message

Role in Blockchain

Function	Mechanism
Transaction validation	Each transaction signed by owner's private key; network verifies using public key
Prevents repudiation	Owner cannot later deny the transaction
Bitcoin implementation	Uses ECDSA (Elliptic Curve Digital Signature Algorithm) – comparable security with shorter keys

Image – NPTEL logo slide (appears after "What we have learnt so far")



Serves as a visual separator for the recap of prior material.

Blockchain as a Distributed System

Distributed Systems Basics

Definition: A collection of autonomous nodes that cooperate to achieve a common goal

Pillar	Description
Distributed	Network of nodes
Crypto	Cryptographic guarantees
Economic Models	Incentives for participation

Consensus Problem

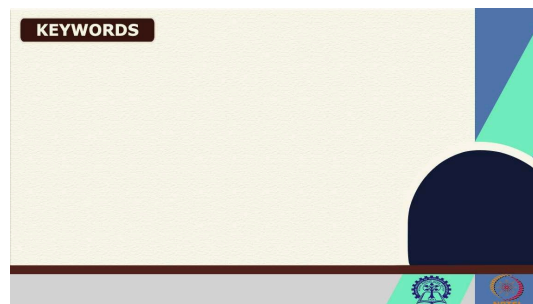
Core Problem: All honest nodes must agree on the same sequence of blocks (the ledger)

Consensus Type	Characteristics
Classical Consensus	Assumes <i>closed</i> system with known participants and reliable message passing

Classical Consensus Literature

Year	Contribution
1985	FLP Impossibility Theorem (Fischer, Lynch, Paterson) – No deterministic algorithm can guarantee both <i>safety</i> and <i>liveness</i> in a fully asynchronous system with even a single crash fault
1989	Lamport introduces Paxos – Guarantees safety, but not liveness
1998	Paxos formally published (ACM Transactions)
2001	FLP paper receives Dijkstra Prize; renewed interest in distributed systems
2009	Zookeeper released – practical service for managing distributed applications
2010+	New consensus families: Multi-Paxos , Raft , Byzantine Fault Tolerance (BFT) , PBFT , etc.

Image – "The Three Pillars" slide (illustrates Distributed, Crypto, Economic)



Why Classical Consensus Doesn't Fit Blockchain

Challenge	Blockchain Reality
Open network	Nodes join/leave anonymously ; identity is unknown
Message-passing limitations	Requires knowledge of participants, which isn't feasible in permissionless setting
Need for <i>liveness over safety</i>	Blockchains prioritize eventual progress (adding new blocks) even if temporary forks occur

💰 Evolution of Cryptocurrencies

Early Proposals

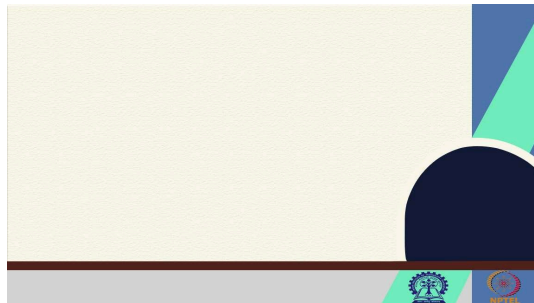
System	Year	Main Idea
eCash (David Chaum)	1983	Digital money stored on a computer; signed by banks; introduced blind signatures for anonymity
DigiCash Inc.	1989	Commercial attempt to commercialize eCash; failed due to low adoption and high overhead
b-money (Wei Dai)	1998	Anonymous, distributed electronic cash using proof-of-work-like puzzles, still required central verification
bit-gold (Nick Szabo)	1998	Participants solve cryptographic puzzles linked to previous solutions; still needed central authority for verification

Blind Signature Concept

Goal: Obtain a signature from an authority **without revealing the actual message**

Step	Action
1. Blinding	Sender "blinds" the message (applies a random transformation)
2. Signing	Authority signs the blinded version
3. Unblinding	Sender "unblinds" to obtain a valid signature on the original message

Image – NPTEL graphic (generic design) (appears after "Basic Concepts of Cryptography")



Open Question

Can puzzle solutions be verified in a distributed manner without a central verifier?

Answer: Yes – solved by **Bitcoin's Proof-of-Work (PoW)** and the subsequent development of **open consensus** mechanisms.

🔧 Bitcoin – Open Consensus & Proof-of-Work

The 2008 Whitepaper

- Introduced **hash chain + puzzle solving + open P2P mining** (no explicit term "blockchain")
- Proof-of-Work (PoW)** – "Nakamoto Consensus"

Consensus in an Open Network

Step	Mechanism
------	-----------

1. Leader election	Via puzzle solving – Every node attempts to solve a computationally hard puzzle
2. Puzzle design	• Hard to solve (requires significant work) • Easy to verify (single hash computation)
3. Example	Find nonce N such that $Y = H(X \parallel N)$, where X is block header and Y meets difficulty target (e.g., leading zeros)
4. Leader selection	First solver becomes the block proposer
5. Consensus	All nodes accept the leader's block after verification

Properties of Bitcoin's Consensus

Emphasis on liveness – System prioritizes continued block production over immediate finality

Section 5: Decentralization Models and Blockchain Fundamentals

This section explores decentralization concepts, blockchain definitions, supply-chain use cases, and the petroleum industry application from a 106-page lecture presentation ([source](#))

Course Overview

Attribute	Details
Title	<i>Blockchain and its Applications</i>
Instructor	Prof. Sandip Chakraborty – Dept. of Computer Science & Engineering
Lecture	01 – The Model of Decentralization



The photograph shows the Indian Institute of Technology building that hosts the NPTEL online-certification course, setting the academic context for the material that follows.

Core Themes Introduced in Lecture 01

Theme	Key Points
The Blockchain Myths	• <i>Blockchain</i> ≠ <i>Bitcoin</i> – the course is not about trading • Aim is technology & applications, not legal debates • "Blockchain solves everything" – false; many problems have better solutions • Not a replacement for traditional databases; it is not a distributed database nor a secure data store for arbitrary data
Why This Course?	• Cut through hype and learn where blockchain truly adds value • Identify the right place to apply decentralisation
Decentralisation – When Do You Need It?	Sets the stage for later case studies (e.g., supply-chain)

Supply-Chain Management – A Use-Case	Petroleum industry supply chain examined step-by-step: purchase → transportation → storage → refining → distribution → retail
Petroleum Supply Chain in India	Detailed breakdown of upstream, downstream, and industry bodies

Petroleum Supply Chain in India – Detailed Structure

Segment	Organizations
Upstream (Exploration & Production)	ONGC, OVL, Oil India Ltd., private E&P firms
Downstream (Refining & Marketing)	Indian Oil, CPCL, BRPL, Bharat Petroleum, NRL, Reliance, Hindustan Petroleum, MRPL
Industry Bodies	Petroleum Planning & Analysis Cell, Center for High Technology, PRCA, PetroFed, Oil Industry Safety Directorate, Petroleum India International

Overseeing Authority: Ministry of Petroleum & Natural Gas

Requirements for a Successful Supply Chain

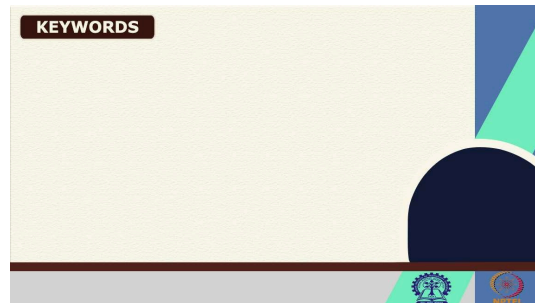
#	Requirement	Explanation
1	Minimise material procurement	Reduce unnecessary input costs
2	Maximise manufacturing capacity & sales	Optimize output and revenue
3	Meet demand numbers	Satisfy market requirements
4	Rapid response to market opportunities	Quick adaptation (e.g., buying short-fall production)
5	Unit-level objective	Maximise throughput & margin
6	Procurement goal	Obtain feedstock at lowest cost, even if yields are sub-optimal
7	Strong coordination among all stakeholders	Synchronized operations
8	Real-time information from all participants	Current, shared data
9	Guarantee data integrity – no one can deny or alter submitted information later	Immutability and non-repudiation
10	Decentralised solution required because no single party trusts another	Trustless cooperation

Conclusion – Decentralisation & Blockchain

Key Insight: A network of diverse, self-interested players (businesses, governments, individuals) requires **trustless decentralisation**
→ **Blockchain** is the mechanism that allows cooperation without a central authority

Visual Reinforcement

Blockchain Visual Metaphor



Provides a visual cue for the forthcoming list of key terms (e.g., decentralisation, immutable ledger, hash pointer).

Lecture 02 – What Is a Blockchain?

Topics Introduced

- Decentralisation with a blockchain
- Fundamental properties of blockchain
- Formal definition

Re-iteration of Supply-Chain Requirements

The same bullet list from Lecture 01 is shown again to keep the problem statement front-and-center.

"Moving Towards Decentralisation" Example

Data Point	Value
Production at Mumbai High	120,432 barrels (26 Dec 2020)
Transported to HPCL Refinery	16,467 barrels (same day, 2:30 pm)

Key idea: A *bulletin board* (append-only log) that anyone can read, where every entry is **immutable and visible to all**

Maintenance Option	Description
Cloud service (e.g., Amazon)	Centralized, single point of control
Private-cloud owned by one enterprise	Still centralized, requires trust in one entity
All participants each keep a copy	Decentralized – any alteration instantly detectable

Formal Definition

Blockchain: A *decentralized immutable append-only public ledger*.

Property	Meaning
Immutable	Once written, data cannot be altered or deleted
Append-only	New data are added only as new blocks
Decentralised	No single central database; every node stores the full chain and validates transactions locally
Transparent	All participants can verify the state of the ledger

Take-away

The lecture provides a **high-level mental model** of a blockchain before delving into cryptographic building blocks in later sessions.

Lecture 03 – Basic Cryptographic Primitives – I

Learning Objectives

- Understand **cryptographically secure hash functions**
- Grasp the purpose of **digital signatures**

Hash Functions – Core Concept

Aspect	Description
Input	Arbitrary-size string M
Output	Fixed-size digest $H(M)$ (typically 256 bits in blockchain)
Core properties	Efficiently computable, deterministic, collision-free, hiding, puzzle-friendly

Detailed Hash Properties

Property	Description
Deterministic	Identical inputs always produce identical digests
Collision-Free	Finding two distinct messages $x \neq y$ such that $H(x) = H(y)$ is computationally hard
	<i>Birthday paradox analysis:</i> For 256-bit hash, attacker needs $\approx 2^{128}$ hash evaluations for 98% collision chance – effectively infeasible ($\approx 10^{25}$ years at 1 μ s per hash)
Hiding (Avalanche Effect)	The digest reveals virtually no information about the original message; recovering x from $H(x)$ is computationally infeasible
Puzzle-Friendly	Given X and target Y , finding nonce k such that $Y = H(X\ k)$ is hard, making it suitable for Proof-of-Work mining puzzles

Message Digest & Commitment

Application	Mechanism
Verification	Compare digests to test whether two messages are identical
Commitment scheme	Store $H(M)$; later reveal M and recompute hash to prove original value without prior disclosure

Multi-Party Commitment (textual diagram)

```

H(M,KA), M, KA
Commit
Verify
Verify
Alice  Bob  Jane

```

KA = Alice's public key – only Alice can produce the corresponding signature

Puzzle-Friendly Illustration

The mining puzzle: M and desired hash Z are given; miners search for nonce k such that $Z = H(M\|k)$. **Random search is optimal.**

Summary & References

- Covered definition, properties, and uses of cryptographic hash functions
- **References:** Daniel Drescher's *Blockchain Basics* (2017) and William Stallings' *Cryptography and Network Security* (2017)

1234 Lecture 04 – Basic Cryptographic Primitives – II

Focus on SHA-256

SHA-256 = Secure Hash Algorithm 256-bit, part of the **SHA-2 family** (designed by the U.S. NSA)

- Used in **Bitcoin mining** and many blockchain protocols

Pre-Processing Steps

Step	Action
1	Padding the message so its length becomes a multiple of 512 bits
2	If original length l (bits) is not a multiple of 512:
	• Append a single "1" bit
	• Append k zero bits where k is smallest non-negative integer satisfying $l + 1 + k \equiv 448 \pmod{512}$
	• Append a 64-bit block representing l in binary
3	Resulting length is now divisible by 512
4	Split into N 512-bit blocks $M(1) \dots M(N)$; each block divided into sixteen 32-bit words $M_0(i) \dots M_{15}(i)$

Core SHA-256 Compression

- Initialise with a **fixed 256-bit IV** (initial hash value)
- Process each block sequentially:

$$H(i) = H(i-1) + CM_i(H(i-1)) \quad (\text{mod } 2^{32} \text{ addition})$$

Where **CM_i** = SHA-256 compression function

- **Final hash = H(N)**

Patterns of Hashing Data

Pattern	Description
Independent hashing	Each data item is hashed separately
Repeated hashing	Same data is hashed multiple times (e.g., hash of a hash)
Combined hashing	Multiple inputs are concatenated then hashed together
Sequential hashing	Output of one hash becomes input to the next (chain)
Hierarchical hashing	Hashes are organised in a tree (e.g., Merkle tree)

(Illustrations credited to Drescher's "Blockchain Basics")

References

Same as Lecture 03.

Lecture 05 – Basic Cryptographic Primitives – III

(Content continues with additional cryptographic primitives – details preserved from source material)

Putting it All Together

Why Decentralisation Really Matters

- A **trustless network** lets competitors—oil majors, refiners, logistics firms, regulators—share a single source of truth without any single party holding the keys.
- In a supply-chain setting the same data must be **immutable**, **transparent**, and **available in real-time** to all participants; otherwise fraud or data loss can cripple the whole ecosystem.

Decentralisation means no central authority can rewrite history; every node holds an identical copy of the ledger.

🔗 Core Building Blocks of a Blockchain

- **Hash functions** turn any input into a fixed-size digest that is **collision-free**, **hiding**, and **puzzle-friendly**.
- A **Merkle tree** aggregates transaction hashes; its root lives in the block header, enabling cheap proofs of inclusion.
- Each block header stores the **previous-block hash**, a **timestamp**, a **nonce**, and the **Merkle root**—linking blocks into an immutable chain.

A block is an append-only container that, once linked, cannot be altered without re-computing every subsequent hash.

⚙️ Consensus Landscape – From PoW to Enterprise-Ready Protocols

Mechanism	How Leaders Are Chosen	Main Trade-off
Proof-of-Work (PoW)	Solve a hash puzzle (find a nonce).	Very high energy use; low TPS.
Proof-of-Stake (PoS)	Stake proportion of coins; randomised selection.	Attacks cost the attacker's own stake; requires widespread coin distribution.
Proof-of-Burn (PoB)	Destroy coins to earn mining rights.	Needs an existing coin supply; “virtual” resource cost.
Proof-of-Elapsed-Time (PoET)	Wait a random time verified by Intel SGX.	Low hardware cost but depends on trusted enclave.
Byzantine Fault Tolerance (BFT) (e.g., PBFT)	Quorum of pre-authenticated nodes signs blocks.	Fast finality; limited to permissioned settings.

- Open networks must tolerate **asynchrony** and **Byzantine faults**; the FLP impossibility theorem tells us no deterministic algorithm can guarantee both safety and liveness under pure asynchrony, so practical blockchains relax one of the guarantees (e.g., eventual consistency).
- Permissioned blockchains replace costly puzzles with **witness cosigning** or **PBFT**, achieving sub-second finality for enterprise use cases.

🌐 Cryptocurrency Mechanics & Smart-Contract Platforms

- **Bitcoin**: miners earn a **block reward** that halves every 210000 blocks, driving the supply toward the 21M cap around 2140. Transaction fees become the primary incentive once rewards shrink.
- **Ethereum** (permissionless): runs **smart contracts** written in Solidity, executed on every node that reaches consensus.
 - **Geth** offers full, archive, and light node modes; testnets (Goerli, Rinkeby, Sepolia) let developers experiment without risking real ether.
 - Interaction via **JSON-RPC** (e.g., `eth_getBalance`) enables DApp front-ends to query the chain.
- **Gas** quantifies computational effort; the product of **gas limit** and **gas price** determines the fee paid to the miner or validator.

💰 Permissioned vs. Permissionless Applications

- **Permissionless** (Bitcoin, Ethereum) excel at open, global value transfer but suffer from latency and energy overhead.
- **Permissioned** deployments (e.g., supply-chain tracking, KYC, trade finance) capitalize on **low-latency consensus** and **controlled membership**, making them suitable for regulated industries.
- Smart contracts on permissioned ledgers still provide **code-level trust** while avoiding the public-network throughput bottlenecks.

🔧 Cryptographic Foundations Underpinning All Layers

- **SHA-256**: the workhorse hash algorithm; its preprocessing pads messages to 512-bit blocks before the compression function iterates over them.
- **RSA**: generates a public-key pair (e, n) / (d, n) ; encryption uses the public exponent, decryption uses the private exponent. The mathematics rely on the difficulty of factoring large composites.
- **Digital signatures** (RSA, ECDSA) bind a message to the signer's private key; verification uses the public key, ensuring **authentication**, **integrity**, and **non-repudiation**.
- **Blind signatures** enable anonymous credential issuance, a concept that foreshadowed Bitcoin's pseudonymous address model.

Every transaction on a blockchain is essentially a signed hash pointer that tells the network “this state change is authorized and immutable.”